RTSVDD Control Chart

Algorithm Technical Documentation

Real-Time SVDD for Multivariate Process Monitoring

Language: Python
Algorithm: RTSVDD Control Chart
Date: March 2026

1. Overview

This document describes the algorithmic implementation of the RTSVDD (Real-Time SVDD) control chart, a novel multivariate statistical process monitoring method that combines Support Vector Data Description (SVDD) with a rolling reference set mechanism and time-weighted test statistics.

The algorithm monitors a multivariate test sequence by maintaining a rolling reference set S0 that is updated online as new observations arrive. At each step, a weighted SVDD is fitted to the combined reference and incoming window, and a time-weighted statistic is computed for the current window. If this statistic exceeds an adaptive control limit derived from the reference set, an alarm is raised; otherwise the reference set is updated by appending the new observation and dropping the oldest.

Module	Role
Data ingestion and normalisation	Load train/test CSVs; apply joint RobustScaler normalisation
Initial SVDD screening (BaseSVDD)	Fit SVDD on training data; select a representative reference set S0 from the inter-quartile distance band
Moving window construction	Prepend tail of S0 to test sequence; slice windows of length Movewindow
Time-weighting schemes	Compute exponentially decaying weights for window observations and reference set
Online monitoring loop	Iterate over test observations: fit weighted SVDD, compute T-statistic, compare to adaptive control limit, update S0

Module	Role
Alarm detection and visualisation	Collect alarm indices; plot T-statistic against control limit

2. Data Ingestion and Normalisation

2.1 Input Files

The pipeline requires two CSV files: `real example-testdata.csv` and `real example-traindata.csv`. The first column of each file is dropped via `.iloc[0:n, 1:]`, treating it as a row-index column. All remaining columns are used as multivariate features.

Variable	Source	Shape after loading	Role
`y_test`	`real example-testdata.csv`	(n_test, p)	Observations to be monitored online
`y_train`	`real example-traindata.csv`	(n_train, p)	Historical in-control reference data for SVDD screening and S0 construction
`R`	Column Confirmedcases of traindata	(n_train,)	Loaded but not used in the monitoring loop; available for external reference

2.2 Joint Robust Normalisation

Training and test data are concatenated row-wise before scaling, so that the same transformation is applied consistently to both sets:

```
All = np.r_[y_test, y_train]
robust = preprocessing.RobustScaler()
All = robust.fit_transform(All)
y_test  = All[:y_test.shape[0], :]
y_train = All[y_test.shape[0]:, :]
```

Fitting the scaler on the combined array prevents data leakage that would arise from fitting separately on training data and then transforming test data with a different scale. `RobustScaler` centres each feature by its median and scales by its interquartile range, making the transformation resistant to outliers in the reference period.

3. Initial SVDD Screening and Reference Set Construction

3.1 Screening SVDD

A preliminary SVDD is fitted on the full normalised training set using an RBF kernel with `C = 1` and `gamma = 0.4`. The distance of every training observation from the SVDD hypersphere centre is then computed:

```
svdd = BaseSVDD(C=1, gamma=0.4, kernel='rbf', display='off')
svdd.fit(y_train)
distances = svdd.get_distance(y_train)   # shape: (n_train,)
```

Each training observation is assigned an index in `sample_ids = np.arange(0, n_train)`, and the (index, distance) pairs are stored in `distance_dict` and sorted by distance in ascending order.

3.2 Inter-Quartile Band Selection

The sorted distances are used to select a representative subset of training observations as the initial reference set. Observations whose distance falls in the band between the 35th and 50th percentiles of the sorted list are retained:

```
n = len(sorted_items)
q1_index = int(n * 0.35)   # lower bound of selection band
q3_index = int(n * 0.50)   # upper bound of selection band
selected_sample_ids = [item[0] for item in sorted_items[q1_index:q3_index]]
```

This band targets observations at moderate distance from the hypersphere centre: close enough to be representative of normal operating conditions, but not so close that they are degenerate support vectors. Observations in the lower 35% (very close to the centre) and upper 50% (outlier-like) are excluded.

3.3 Random Subsampling to S0

From the selected band, `S0 = 100` observations are drawn uniformly at random without replacement to form the initial reference set `y_S0`:

```
S0 = 100
S0_0 = y_train_pro.shape[0]   # size of the inter-quartile band
row_indices = np.random.choice(S0_0, size=S0, replace=False)
y_S0 = y_train[row_indices, :]   # shape: (100, p)
```

Note: The subsampling draws row indices from `y_train_pro` (the band-filtered subset) but applies them to index `y_train` directly. Because `valid_indices` are positions within the original training array, this correctly retrieves the selected observations from `y_train`.

4. Moving Window Construction

The test sequence `X = y_test` is prepended with the last `Movewindow - 1 = 4` rows of `y_S0` to

form the extended sequence `X_w`. This ensures that the first test observation can be evaluated within a full window of length `Movewindow = 5`:

```
Movewindow = 5
X_w = np.r_[y_S0[y_S0.shape[0]-Movewindow+1:, :], X]
# X_w shape: (n_test + Movewindow - 1, p)
```

At iteration `i` of the monitoring loop, the current window is `X_w[i : Movewindow+i, :]` (rows i through i+Movewindow−1), and the newest single observation entering the window is `X_w[Movewindow+i-1 : Movewindow+i, :]`.

5. Time-Weighting Schemes

Two separate weight vectors are constructed: one for the moving window observations used in the T-statistic, and one for the reference set S0 observations used when fitting the SVDD.

5.1 Window Weight Vector

The weight assigned to the k-th observation in the moving window (ordered from oldest to newest, $k = 0, \ldots, \mathrm{Movewindow}-1$) is:

$$\mathrm{w_k} = 1 \,/\, (b \cdot (\mathrm{Movewindow} - 1 - k) + 1), \quad b = 0.1$$

The vector is then normalised to unit sum: $\hat{w}_k = w_k \,/\, \Sigma\, w_k$. With `b = 0.1` and `Movewindow = 5`, the most recent observation (k = Movewindow−1) receives weight $1/(0{\cdot}1{+}1) = 1$ before normalisation, while the oldest (k = 0) receives $1/(0.1{\times}4{+}1) = 0.714$. The resulting normalised weights decay from newest to oldest.

```
b = 0.1
weight_f = [[1/(b*(Movewindow-1)+1)]]          # weight for oldest observation
for i in range(Movewindow-1):
    weight_t = [[1/(b*(Movewindow-i-2)+1)]]    # weight for newer observations
    weight_f = np.r_[weight_f, weight_t]
weight_nol = weight_f / np.sum(weight_f)       # normalised, shape (Movewindow, 1)
```

5.2 Reference Set Weight Vector

A separate weight vector is constructed for the combined reference set `y_combine = np.r_[y_S0, New_X_w]`, which has $S0 + \mathrm{Movewindow} = 105$ rows. With `b = 0`, the formula $1 \,/\, (b \cdot k + 1)$ reduces to 1 for all k, yielding a uniform weight vector of length $S0 + \mathrm{Movewindow}$. All observations in `y_combine` therefore receive equal weight when fitting the SVDD.

```
b = 0
# With b=0, all weights equal 1/(0+1)=1 before normalisation
weight_f_S0 = [[1/(b*(y_S0.shape[0]+Movewindow-1)+1)]]
for i in range(y_S0.shape[0]+Movewindow-1):
```

```
    weight_t_S0 = [[1/(b*(y_S0.shape[0]+Movewindow-i-2)+1)]]
    weight_f_S0 = np.r_[weight_f_S0, weight_t_S0]
# weight_f_S0 shape: (S0 + Movewindow, 1), all entries = 1
```

Note: The weight vector `weight_f_S0` is passed directly to `svdd.fit()` without normalisation. The length of this vector equals `y_S0.shape[0] + Movewindow`. Because `y_S0` is updated at the end of each non-alarm iteration by appending one observation and removing the oldest, its row count remains fixed at `S0 = 100` throughout the loop, and the combined length stays constant at 105.

6. Online Monitoring Loop

6.1 Loop Structure

The monitoring loop iterates over `i = 0, 1, …, X_w.shape[0] - Movewindow`, which equals the number of test observations. At each step, the algorithm constructs the current window, fits a weighted SVDD, computes the T-statistic, and decides whether to raise an alarm or update the reference set.

```
for i in range(X_w.shape[0] - Movewindow + 1):
    New_X   = X_w[Movewindow+i-1 : Movewindow+i, :]   # newest single observation
    New_X_w = X_w[i : Movewindow+i, :]                # current window (Movewindow
rows)
    y_combine = np.r_[y_S0, New_X_w]                  # reference + window
```

6.2 Weighted SVDD Fitting

At each iteration, a new SVDD is fitted on `y_combine` using the uniform reference-set weights `weight_f_S0`:

```
svdd = BaseSVDD(C=0.05, gamma=0.8, kernel='rbf', display='off')
svdd.C = 0.05
svdd.fit(y_combine, weight=weight_f_S0)
```

6.3 Adaptive Control Limit

The control limit for the current iteration is derived from the distance distribution of the reference set `y_S0` under the newly fitted SVDD:

```
distance_S0 = svdd.get_distance(y_S0).A.flatten()   # distances of S0 under current
SVDD
R_S0 = np.quantile(distance_S0, 1 - a)              # (1-a)-quantile, a = 0.05
radius_final = R_S0
```

The significance level `a = 0.05` sets the control limit at the 95th percentile of the reference-set distances under the current SVDD. This is an adaptive limit: it is recomputed at every step using the updated SVDD and the current reference set, so it tracks changes in the local geometry of the data space.

6.4 Time-Weighted T-Statistic

The test statistic for the current window is a weighted sum of the SVDD distances of the `Movewindow` window observations:

$$\mathrm{T_new} = \hat{\mathrm{w}}^{\mathrm{T}} \cdot \mathrm{d}(\mathrm{New_X_w})$$

```
TSVDD_new = np.dot(weight_nol.T, svdd.get_distance(New_X_w))[0, 0]
```

where `weight_nol` is the normalised window weight vector of shape (Movewindow, 1) and `svdd.get_distance(New_X_w)` returns the SVDD distance of each window observation. The dot product yields a scalar T-statistic that up-weights recent observations relative to older ones within the window.

6.5 Alarm Decision and Reference Set Update

The T-statistic is compared to the adaptive control limit. The outcome determines both the alarm flag and whether the reference set is updated:

```
if TSVDD_new > radius_final:          # OUT-OF-CONTROL: raise alarm
    radius_f.append(radius_final)
    TSVDD_f.append(TSVDD_new)
    n_alarm += 1
    # S0 is NOT updated; reference set is frozen

elif TSVDD_new <= radius_final:       # IN-CONTROL: update reference set
    radius_f.append(radius_final)
    TSVDD_f.append(TSVDD_new)
    y_S0 = np.r_[y_S0, New_X]        # append newest observation
    y_S0 = np.delete(y_S0, 0, axis=0) # drop oldest observation
```

Condition	Action on S0	Interpretation
T_new > radius_final (alarm)	S0 unchanged (frozen)	Current observation is out-of-control; reference set is not contaminated with anomalous data
T_new ≤ radius_final (in-control)	Append New_X; drop oldest row	Current observation is consistent with normal conditions; S0 rolls forward by one step

The rolling update preserves `y_S0.shape[0] = S0 = 100` at all times. By freezing the reference set during alarms, the algorithm prevents fault propagation: anomalous observations are never incorporated into the reference distribution, ensuring that the control limit remains anchored to in-control behaviour.

7. Output Collection

After the loop, the per-step control limits and T-statistics are converted to column vectors, and the alarm indices are extracted:

```
radius_f = np.array([radius_f]).T          # shape: (n_test, 1)
TSVDD_f  = np.array([TSVDD_f]).T           # shape: (n_test, 1)
n_alarm_indicate = np.where(TSVDD_f > radius_f)[0][:]   # alarm step indices
```

Output Variable	Shape	Content
`radius_f`	(n_test, 1)	Adaptive control limit at each test step
`TSVDD_f`	(n_test, 1)	Time-weighted T-statistic at each test step
`n_alarm`	scalar	Total count of alarm events
`n_alarm_indicate`	(n_alarm,)	Array of step indices where TSVDD_f > radius_f

8. Visualisation

The function `plot_distance(radius, distance)` renders the T-statistic trajectory against the adaptive control limit as a time-series chart.

Element	Style	Represents
Control limit	Red solid line, linewidth 3	`radius_f`
T-statistic	Black dotted line with circular markers	`TSVDD_f`

The x-axis tick labels are hardcoded to map integer indices `[0, 49, 99, 149, 199, 249, 299]` to the date strings `['1-Mar', '20-Apr', '9-Jun', '29-Jul', '17-Sept', '6-Nov', '26-Dec']`. These tick positions and labels are specific to the reference dataset used in the original study and should be adjusted for other datasets.

9. Hyperparameter Summary

Parameter	Value	Where Used	Role
C (screening)	1	Initial SVDD on y_train	Penalty parameter; C=1 allows no outliers in the screening fit
gamma (screening)	0.4	Initial SVDD on y_train	RBF kernel width for screening
q1 percentile	0.35	IQR band selection	Lower bound of representative distance band

Parameter	Value	Where Used	Role
q3 percentile	0.50	IQR band selection	Upper bound of representative distance band
S0	100	Reference set size	Fixed number of observations in the rolling reference set
Movewindow	5	Window width	Number of observations in the time-weighted test statistic
b (window weights)	0.1	Window weight vector	Decay rate for within-window time weighting; b=0.1 yields mild recency bias
b (S0 weights)	0	Reference set weight vector	b=0 yields uniform weights across all S0+Movewindow observations
C (monitoring)	0.05	Per-step SVDD in loop	Tighter penalty for local monitoring SVDD
gamma (monitoring)	0.8	Per-step SVDD in loop	Narrower RBF kernel for local monitoring SVDD
a (significance)	0.05	Control limit quantile	Control limit set at (1-a) = 95th percentile of S0 distances

10. Dependencies and Execution

10.1 Required Packages

Package	Usage
numpy	Array operations, random sampling, quantile computation
pandas	CSV ingestion
src.BaseSVDD	SVDD fitting and distance computation (custom/local module)
sklearn.preprocessing.RobustScaler	Joint normalisation of train and test data
matplotlib / pylab	Control chart visualisation; custom x-axis tick labels
time	Wall-clock timing of the full pipeline

10.2 Execution

Place `real example-traindata.csv` and `real example-testdata.csv` in the working directory, ensure the `src/BaseSVDD.py` module is accessible via `sys.path`, and execute:

```
python RTSVDD.py
```

References

[1] Tax, D. M. J., & Duin, R. P. W. (2004). Support vector data description. *Machine Learning*, 54(1), 45–66.

[2] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.